L35 — Dijkstra's Algorithm

Unit: 3 | **Chapter:** Graphs | **BT Level:** BT5 | **CO:** CO3, CO4

Learning Objectives

- Explain the greedy strategy behind Dijkstra's algorithm
- Implement Dijkstra's algorithm using a min-heap (priority queue)
- Trace the algorithm on a weighted directed graph
- Reconstruct the shortest path and analyze time complexity

---

1. Problem Statement

Single-Source Shortest Path: Given a weighted graph and a source vertex, find the shortest distance from the source to every other vertex.

Constraint: All edge weights must be **non-negative** (Dijkstra fails on negative edges — use Bellman-Ford instead).

---

2. Algorithm Intuition

Dijkstra's is a **greedy algorithm**:

1. Maintain a distance array `dist[]` initialized to ∞ for all vertices except source (dist[source] = 0).
2. Use a **min-priority queue** to always process the vertex with the current minimum distance.
3. For each extracted vertex u, **relax** all its outgoing edges: if `dist[u] + w(u,v) < dist[v]`, update `dist[v]`.
4. A vertex is "finalized" once extracted from the priority queue.

Greedy choice: Once a vertex is extracted, its shortest distance is final (since all edge weights ≥ 0, no future path can be shorter).

---

3. Algorithm

```
Algorithm Dijkstra(graph, source):
  dist[v] = ∞ for all v
  dist[source] = 0
  pq = min-priority queue containing (0, source)
  parent[v] = None for all v

  while pq is not empty:
      (d, u) = extract_min(pq)
      if d > dist[u]: continue   // stale entry
```

```
      for each (v, weight) in neighbors(u):
          new_dist = dist[u] + weight
          if new_dist < dist[v]:
              dist[v] = new_dist
              parent[v] = u
              push (new_dist, v) into pq

  return dist, parent
```

---

4. Implementation

```
import heapq

def dijkstra(graph, source):
    """
    graph: {vertex: [(neighbor, weight), ...]}
    Returns (distances, parent) dictionaries.
    """
    # Initialize distances
    dist = {v: float('inf') for v in graph}
    dist[source] = 0
    parent = {v: None for v in graph}

    # Min-heap: (distance, vertex)
    pq = [(0, source)]

    while pq:
        d, u = heapq.heappop(pq)

        # Skip stale entries
        if d > dist[u]:
            continue

        for v, weight in graph.get(u, []):
            new_dist = dist[u] + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                parent[v] = u
                heapq.heappush(pq, (new_dist, v))

    return dist, parent


def shortest_path(graph, source, target):
    """Reconstruct shortest path from source to target."""
    dist, parent = dijkstra(graph, source)

    if dist[target] == float('inf'):
        return None, float('inf')
```

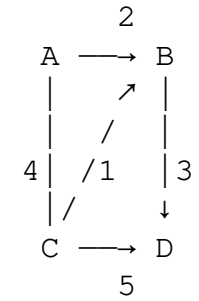
```
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()

    return path, dist[target]
```

---

5. Detailed Trace

Graph:



Adjacency list:

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3)],
    'C': [('B', 1), ('D', 5)],
    'D': []
}
```

Source: A

Initial: `dist = {A:0, B:∞, C:∞, D:∞}`, `pq = [(0, A)]`

Step	Extract	dist[u]	Relax	Updated dist	pq
1	(0, A)	0	A→B: $0+2=2 < \infty$; A→C: $0+4=4 < \infty$	{A:0, B:2, C:4, D:∞}	[(2,B), (4,C)]
2	(2, B)	2	B→D: $2+3=5 < \infty$	{A:0, B:2, C:4, D:5}	[(4,C), (5,D)]
3	(4, C)	4	C→B: $4+1=5 > 2$ (no update); C→D: $4+5=9 > 5$ (no update)	{A:0, B:2, C:4, D:5}	[(5,D)]
4	(5, D)	5	No neighbors	unchanged	[]

Final distances from A:

- A → A: 0
- A → B: 2 (path: A → B)
- A → C: 4 (path: A → C)
- A → D: 5 (path: A → B → D)

6. Full Example with Path Reconstruction

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3)],
    'C': [('B', 1), ('D', 5)],
    'D': []
}

dist, parent = dijkstra(graph, 'A')
print("Distances:", dist)
# {'A': 0, 'B': 2, 'C': 4, 'D': 5}

path, length = shortest_path(graph, 'A', 'D')
print(f"Shortest path A→D: {' → '.join(path)}, length={length}")
# Shortest path A→D: A → B → D, length=5
```

---

7. Why Dijkstra Fails on Negative Edges

```
A −(1)→ B −(−10)→ C
A −(5)→ C
```

Once B is extracted with dist[B] = 1, C is updated to dist[C] = 1 + (-10) = -9.
But after A→C is processed, dist[C] might be set to 5 first and incorrectly finalized.
Use Bellman-Ford for graphs with negative edges.

---

8. Complexity Analysis

Implementation	Time Complexity
Simple array (extract-min by scanning)	$O(V^2)$
Binary heap priority queue	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

For sparse graphs ($E \approx V$): binary heap is optimal.
For dense graphs ($E \approx V^2$): array-based is comparable.

Standard answer: $O((V + E) \log V)$ using a binary min-heap.

---

9. Applications

Application	Graph Model
GPS navigation (shortest road route)	Road network with distances
Network routing (OSPF protocol)	Router network with latencies
Currency arbitrage detection	Currency exchange graph

Game AI pathfinding	Grid/tile map
Social network (shortest connection)	Friend graph

Summary

- Dijkstra's finds shortest paths from a source in **O((V + E) log V)** using a min-heap.
- Greedy: always process the vertex with the current minimum distance.
- **Requires non-negative edge weights** — use Bellman-Ford for negative edges.
- Reconstruct paths by tracing parent pointers back from destination to source.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 24.3 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 8.3 (Springer, 2020)